Hands-on: Document an R Package with Roxygen2

Overview

In this 30 min session you will:

1. Clone a simple R package from GitHub.
2. Create a new Git branch for documentation work.
3. Explore package structure.
4. Add **Roxygen2** documentation.
5. Generate documentation files (**NAMESPACE**, **man/**).
6. Install and test the package.
7. Push changes to GitHub and (optionally) open a pull request.

1.1 Clone the repository

In Terminal, run:

```
1 git clone https://github.com/jaiheechoi/DemoPackage.git
```

1.2 Open the project in RStudio.

You should see a package with this structure:

```
DemoPackage/
   DESCRIPTION
   NAMESPACE
   R/
       euclidean_distance.R
      plot_xy.R
   man/
   DemoPackage.Rproj
```

The package functions are already written and working. **Your job is to add documentation.**

2.1 Create a branch for your work

In Terminal, use:

```
git checkout -b <your-branch-name>
```

Confirm you are on the correct branch:

```
git branch
```

You should see * <your-branch-name>.

3. Explore the package

Open the files in **R/** and inspect the functions:

- **euclidean_distance()**
- **plot_xy()**

Each function works, but the documentation is missing or incomplete.

Take a few minutes to read through each function and identify:

- What the function does
- What inputs it takes
- What it returns

---

4. Add Roxygen2 documentation

Add a Roxygen2 block above the functions:

- euclidean_distance()
- plot_xy()

As you write your Roxygen block, pay attention to:

a. **@param** (function inputs)

- Each input to your function must be described using **@param**
- You should explain:
  - what the argument is
  - its type (numeric vector, matrix, etc.)
  - any important constraints (e.g., "same length as y")

b. **@return** (output)

- This describes what the function gives back after it runs.
- It is just as important as **@param**, because users need to know what to expect.

As you write it, be explicit about:

- the type of output (numeric, vector, matrix, list, data frame, etc.)
- the structure (single value vs multiple values)
- what the values represent

Example:

```
#' k-Nearest Neighbors Prediction
#'
#' Computes predictions for a set of test observations using the k-nearest neighbors (kNN) al
#' The function supports both an R implementation and a C++ backend (`knn_pred_cpp`) for fast
#'
#' @param train_x A numeric matrix or data frame of training predictors (features), with rows
#' @param train_y A numeric vector of training responses (target values).
#' @param test_x A numeric matrix or data frame of test predictors to make predictions for.
#' @param k An integer specifying the number of nearest neighbors to use.
#' @param method A character string specifying the implementation method. `"cpp"` (default) u
#'
#' @return A list with two components:
#' \describe{
#'   \item{preds}{A numeric vector of predicted values for each test observation.}
```

```
#'   \item{ids}{A numeric vector containing the indices of the k nearest neighbors for each t
#' }
#'
#' @examples
#' # Example with random data
#' set.seed(123)
#' train_x <- matrix(rnorm(50), nrow = 10)
#' train_y <- rnorm(10)
#' test_x <- matrix(rnorm(20), nrow = 4)
#' k <- 3
#' result <- knn_pred(train_x, train_y, test_x, k)
#' result$preds
#' result$ids
#'
#' @export
knn_pred <- function(train_x, train_y, test_x, k) {
  ...
}
```

5.1 Generate Documentation

In R, run:

```
devtools::document()
```

This will:

- update the **NAMESPACE**
- generate **.Rd** help files in **man/**

Check that new documentation files appear in **man/**.

5.2 Update `DESCRIPTION`

Open the **DESCRIPTION** file and make sure the package metadata is filled in correctly.

At minimum, check:

- package name

- title
- description
- author
- **Imports**

Because **plot_xy()** uses **ggplot2**, make sure **ggplot2** appears in **Imports**:

```
Imports: ggplot2
```

Save the file after making changes.

5. Install and load your package

In R, run:

```
devtools::install()
library(DemoPackage)
```

6.1 Test your functions

Try each documented function:

```
euclidean_distance(c(1, 2), c(4, 6))
```

```
plot_xy(1:10, (1:10)^2)
```

6.2 Test the help system

Now test whether your documentation works:

```
?euclidean_distance
?plot_xy
```

The Help pane should open and display:

- Title of the function
- Short description of what the function does
- Arguments (@param entries)
- Return value (@return)

- Examples (if provided)

If the documentation was written correctly using Roxygen2, all of this information should appear automatically.

7.1 Update your changes back to Github

Commit your changes. In Terminal, run:

```
git add .
git commit -m "Add roxygen documentation"
```

Push your branch to GitHub

```
git push origin <your-branch-name>
```

7.2 Optional: Create a Pull Request

Go to GitHub and:

- Open your repository
- Click “Compare & pull request”
- Merge <your-branch-name> into main

Important: In this workshop, you will not actually merge into **main** unless you are the repo owner or have write permissions. The goal is to practice the pull request workflow, not necessarily complete the merge.

Next steps

- Add one new function and document it yourself.